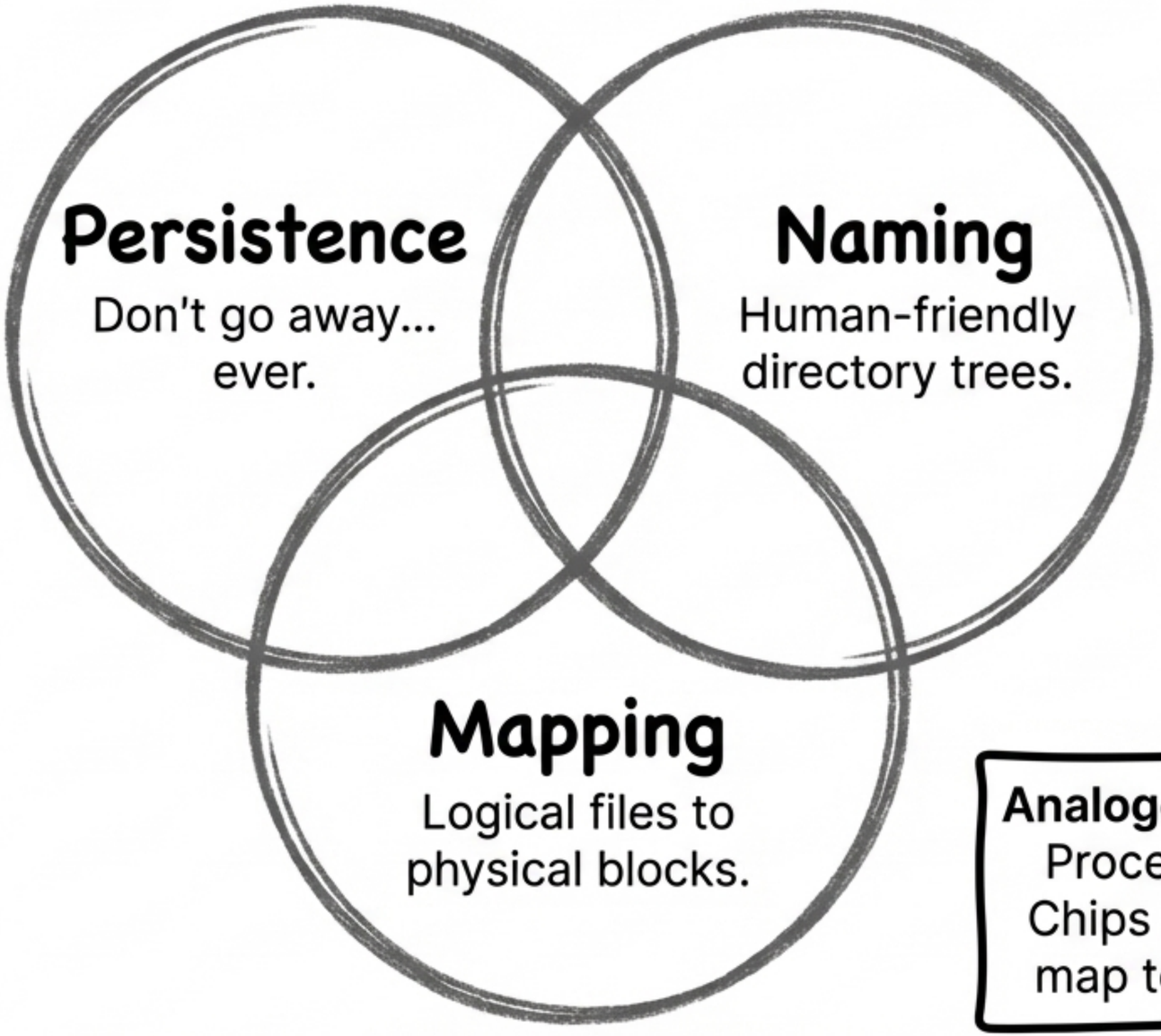
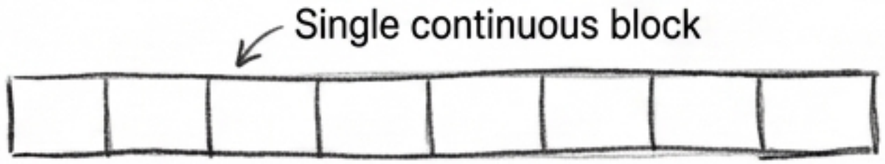
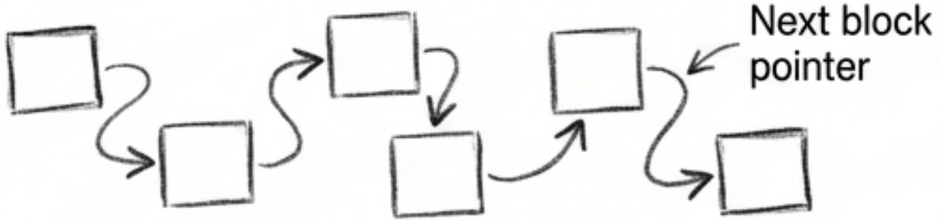
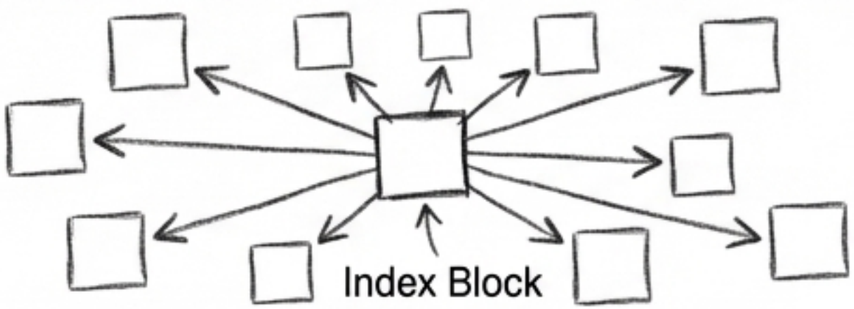
The Three Pillars of the File System



Analogous to Virtual Memory: Processes map to Memory Chips via Page Tables. Files map to the Disk via Inodes.

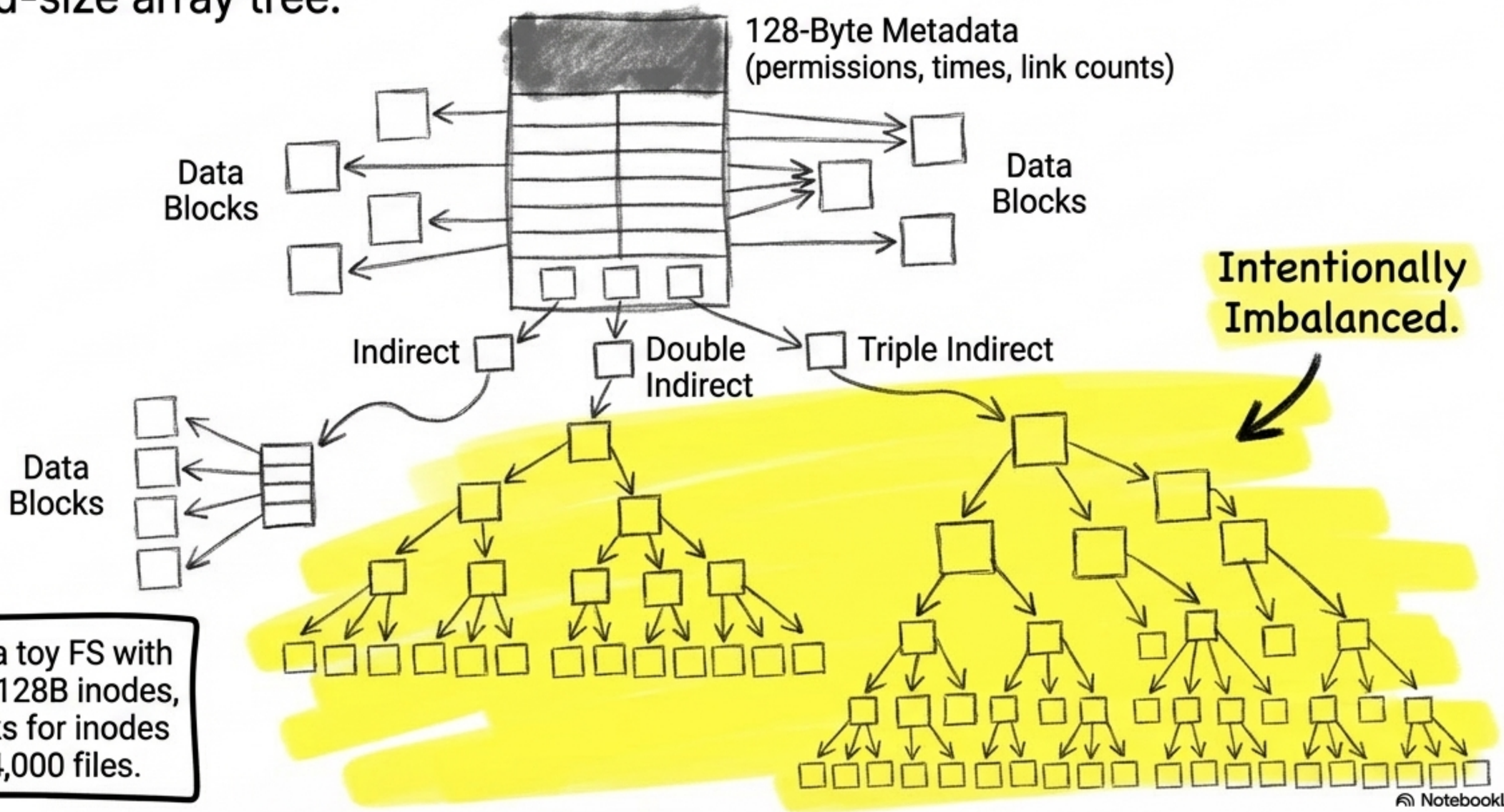
Candidate Designs: How to Allocate Blocks?

Method	Visual Model	Pros & Cons
Contiguous Allocation		**Pros:** Fast sequential & random access. **Cons:** Severe fragmentation. (What if file needs 7 sectors but only 5 are free?)
Linked Allocation		**Pros:** No fragmentation. **Cons:** Random access is a disaster; pointers waste block space.
Indexed Allocation		**Pros:** Solves fragmentation, supports random access. **Cons:** Metadata overhead (storing the array).

Conclusion: The choice depends heavily on workload parameters—most files are small, but large files consume the most disk space.

The Unix Inode: The Indexed Solution

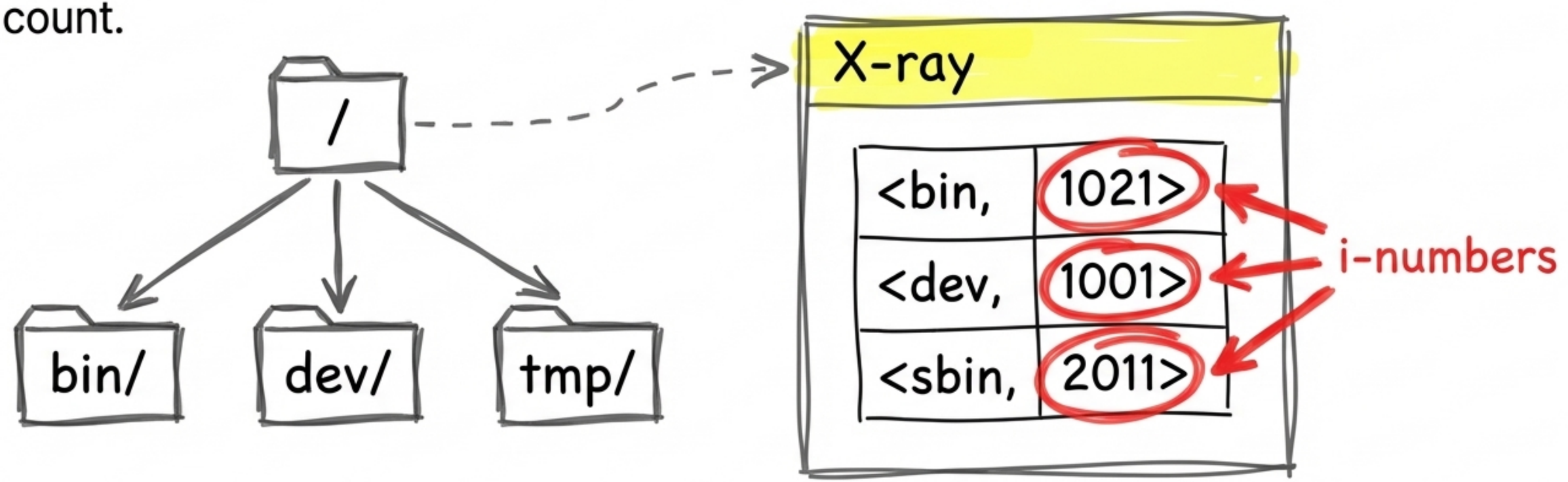
To support massive files without wasting space on small ones, the Unix inode uses a fixed-size array tree.



Napkin Math: In a toy FS with 512B blocks and 128B inodes, using 1000 blocks for inodes yields exactly 4,000 files.

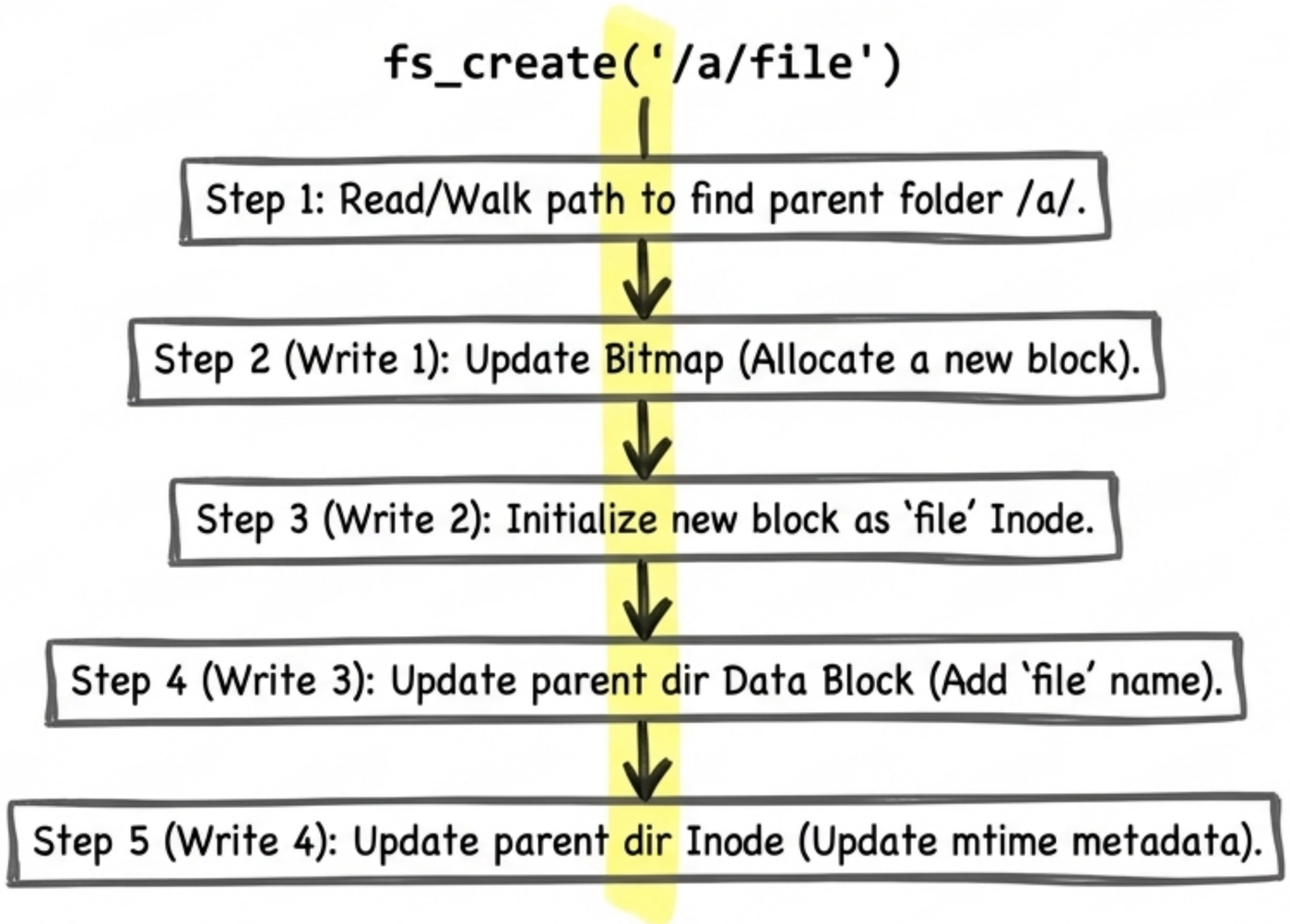
Directories: Mapping Names to Numbers

Directories are just standard files stored on disk. Instead of raw user data, their data blocks contain a simple list of <name, i-number> pairs. A hard link just creates multiple directory entries pointing to the same i-number reference count.



The Mechanics of fs_create

Creating a single empty file is not one operation. It requires a minimum of four distinct block writes across different physical areas of the disk.



The Threat: Disconnected Reality

Writing to one 512B block is guaranteed atomic by hardware.
But multi-block updates are vulnerable.

Exist

- Parent directory has a map entry for it.
- Data exists on a block somewhere
- Inode exists

Step 1: Read/Walk path to find parent folder /a/.

Step 2 (Write 1): Update Bitmap (Allocate a new block). (Inode)

Step 3 (Write 2): Initialize new block as 'file' Inode.

POWER FAILURE

Inode initialized, but parent directory doesn't know it exists.

Step 4 (Write 3): Update parent dir Data Block (Add 'file' name).

Step 5 (Write 4): Update parent dir Inode (Update mtime metadata).

Consequences of crashes between steps:

- Crash 1 & 2: Lost blocks (memory leak or disk).
- Crash 2 & 3: Garbage data in file.
- Crash 3 & 4: Unreachable "orphan" file.

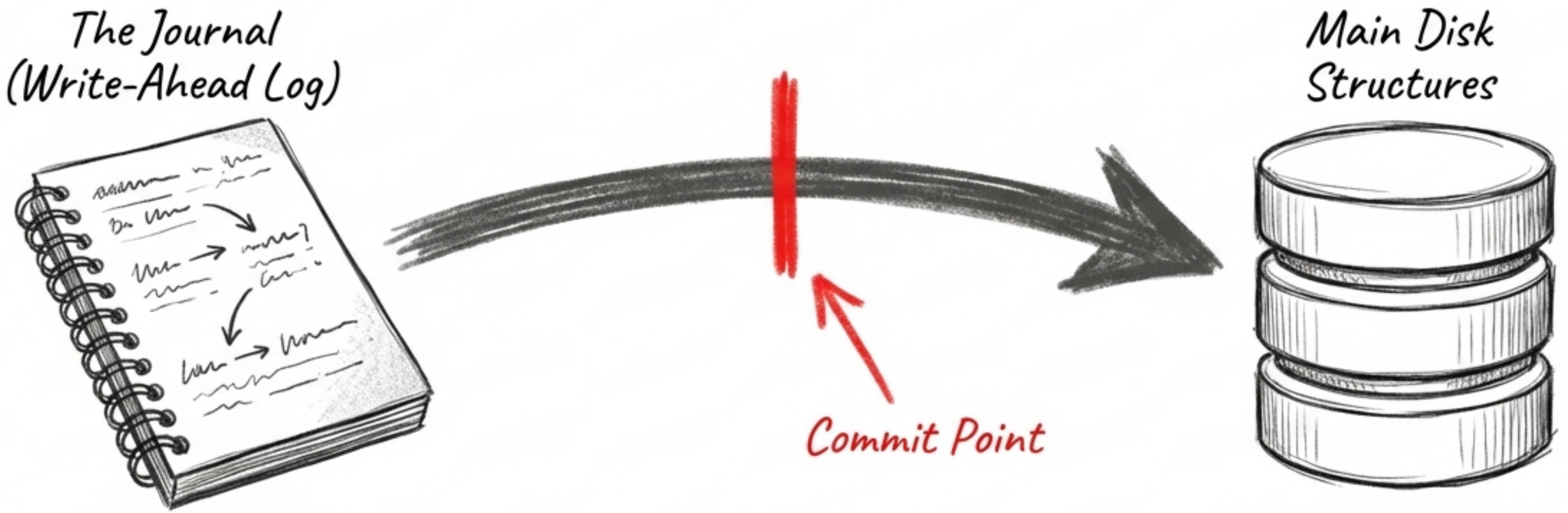
* Writing data to a block

Crash Consistency & Journaling

Golden Rule: “Never modify the only copy.” — Saltzer-Kaashoek

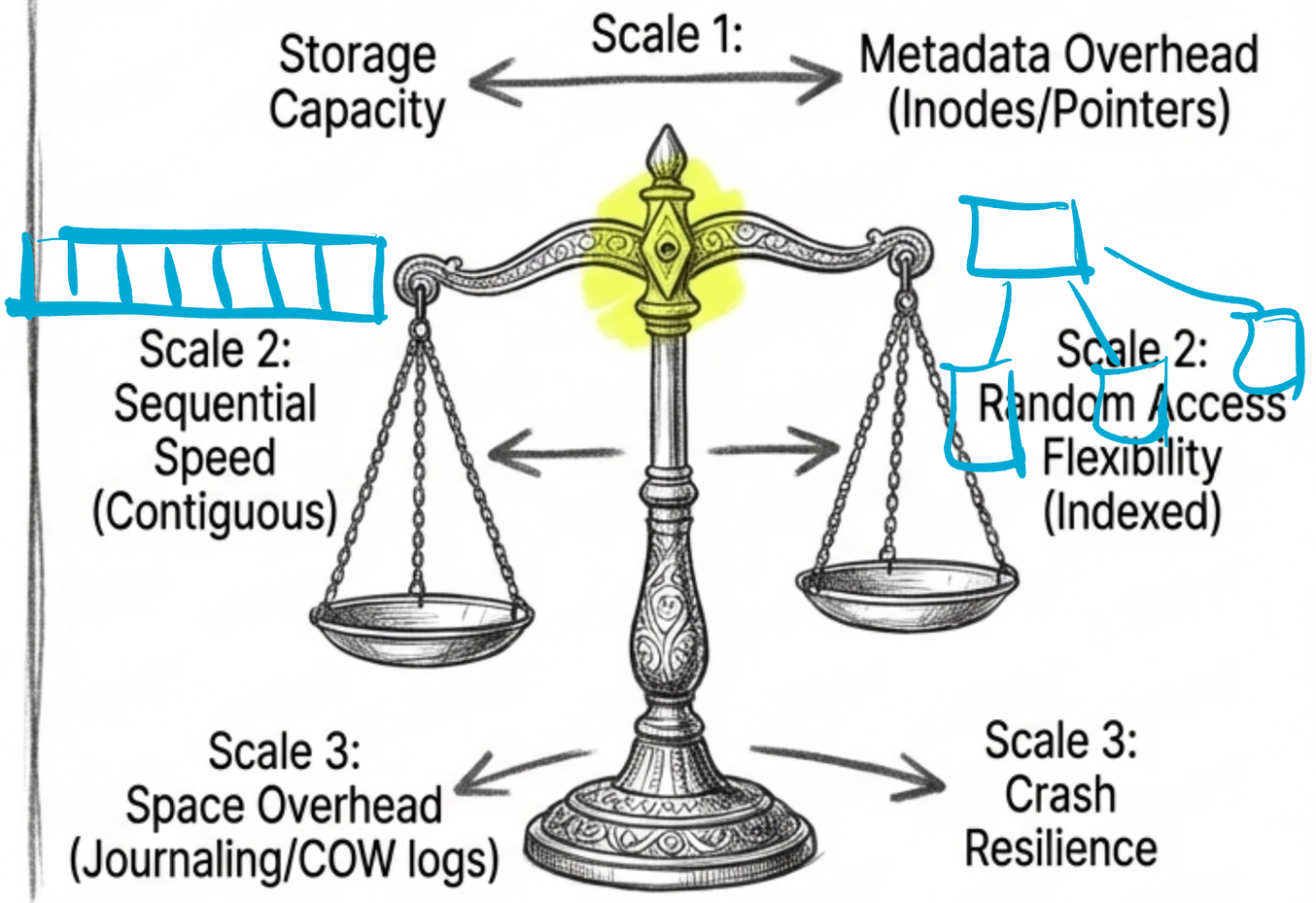
To survive arbitrary crashes, file systems treat operations as **atomic transactions**.

We write a **Redo/Undo log** to the journal first. Only when the log reaches the ‘**Commit Point**’ (no turning back) do we apply the **un-interruptible** operations to the main disk structures.



Core Takeaways & The Architecture of Tradeoffs

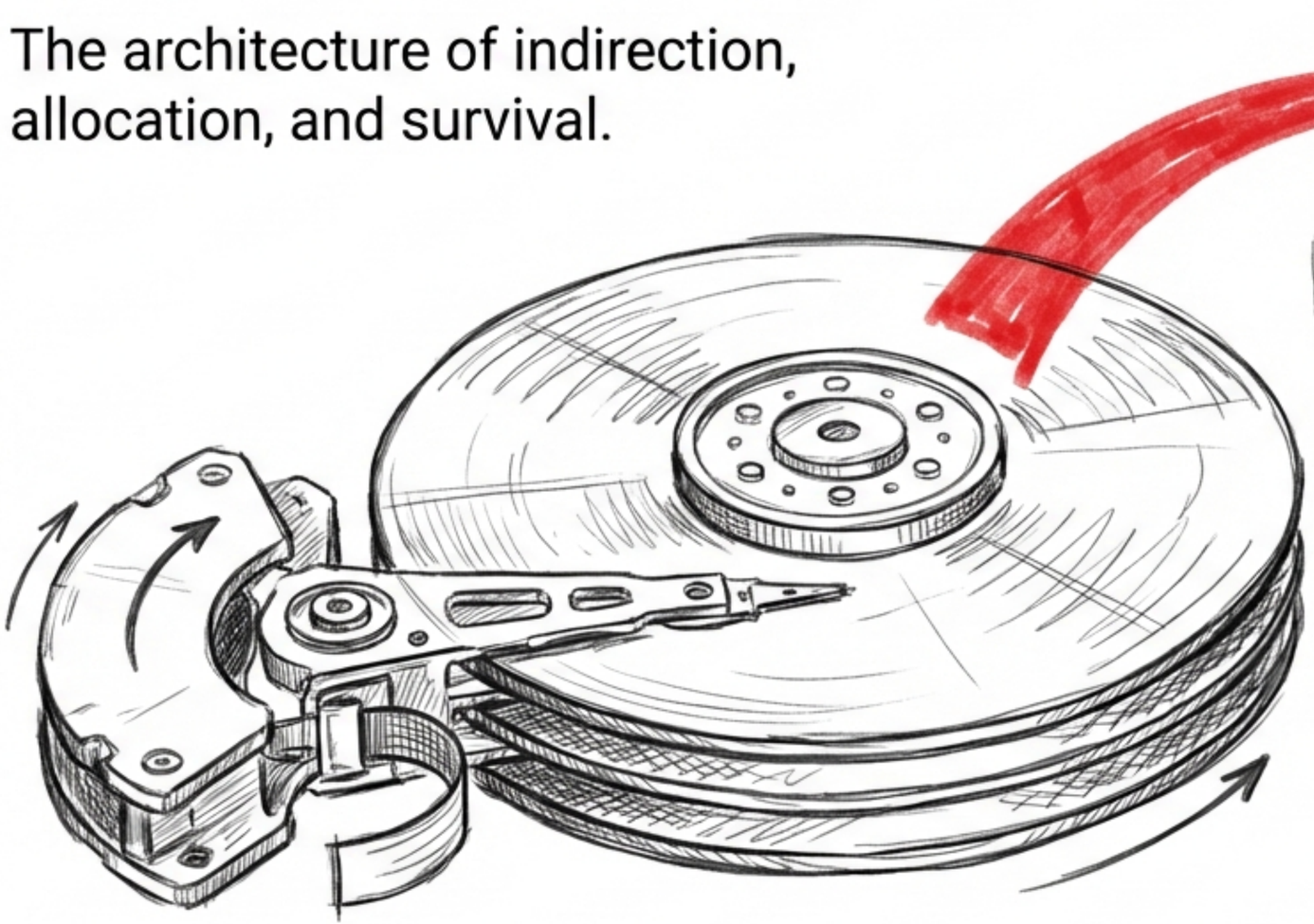
1. **Indirection Solves Everything:**
   FTLs hide physical wear;
   Inodes hide sector mapping;
   Directories hide i-numbers.
2. **Atomicity is Mandatory:**
   Multi-step disk writes must be treated as transactions to survive reality.

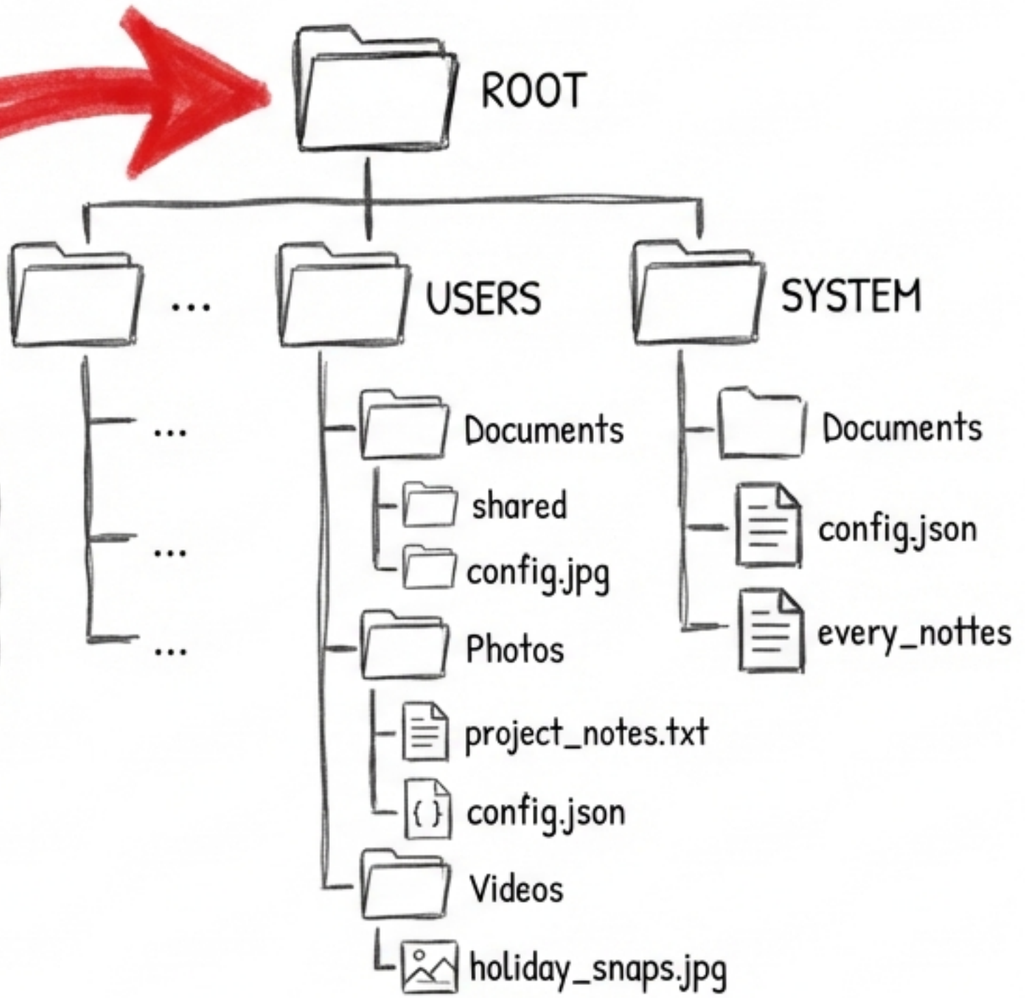


Conclusion: File system design is the art of balancing physical constraints against the human demand for speed, size, and absolute safety.

Intro to File Systems: From Platters to Persistence

The architecture of indirection, allocation, and survival.

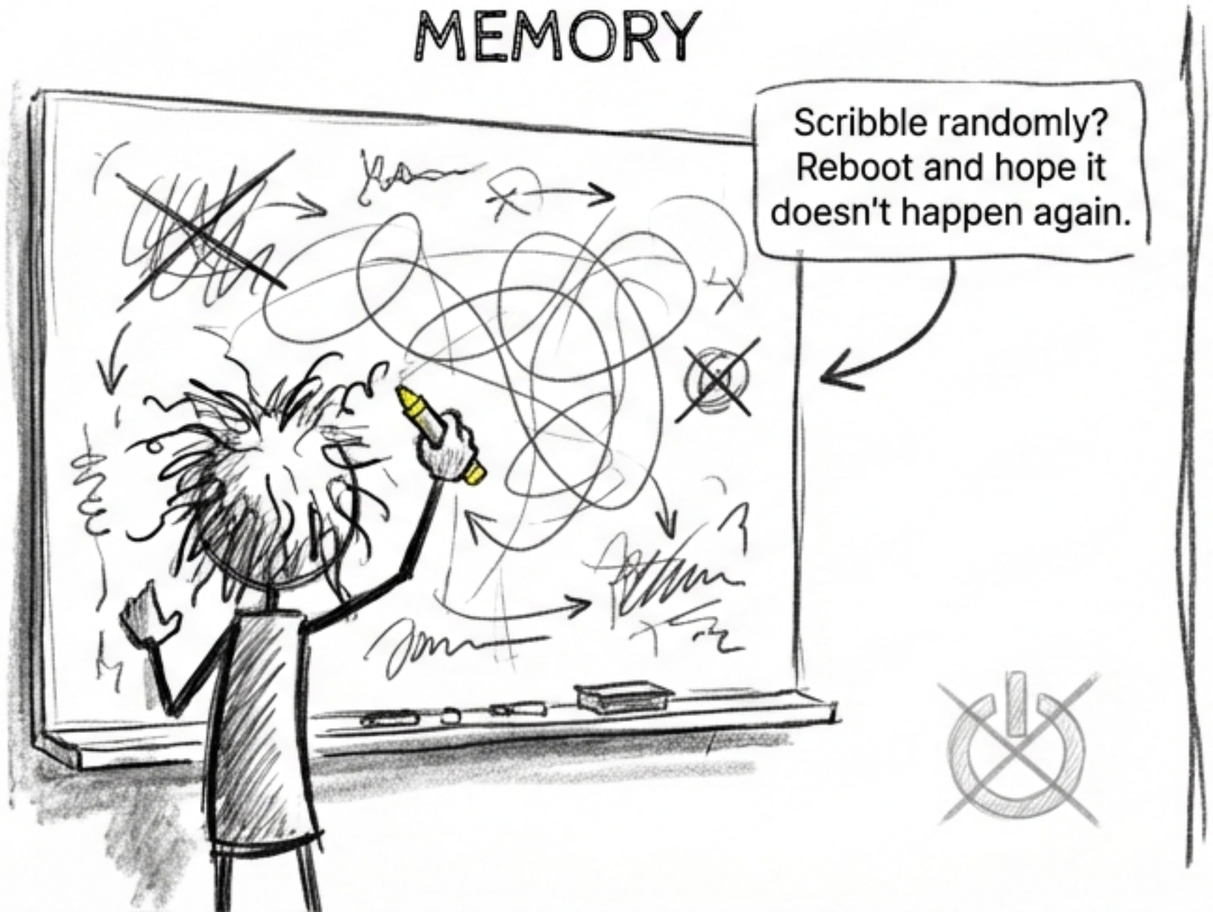


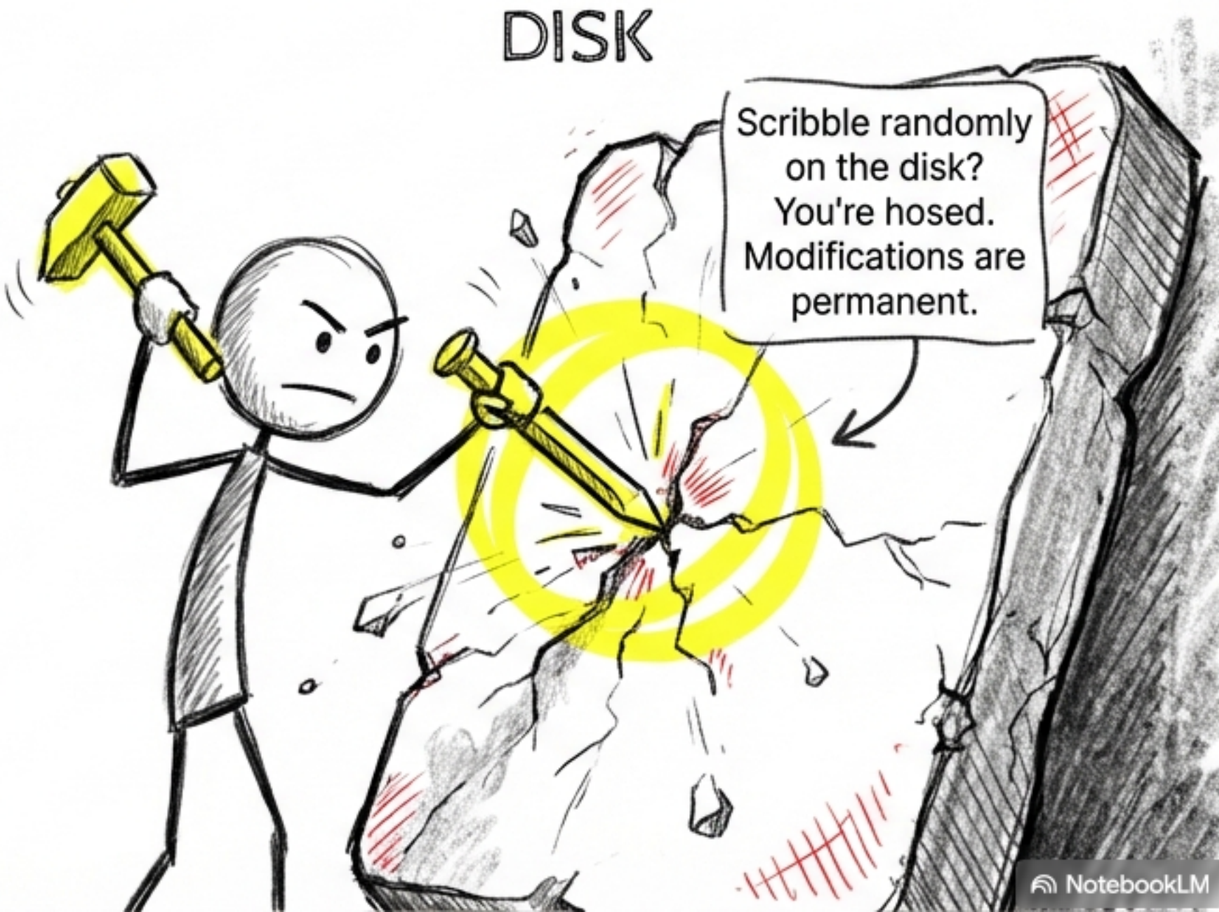


The Persistence Problem

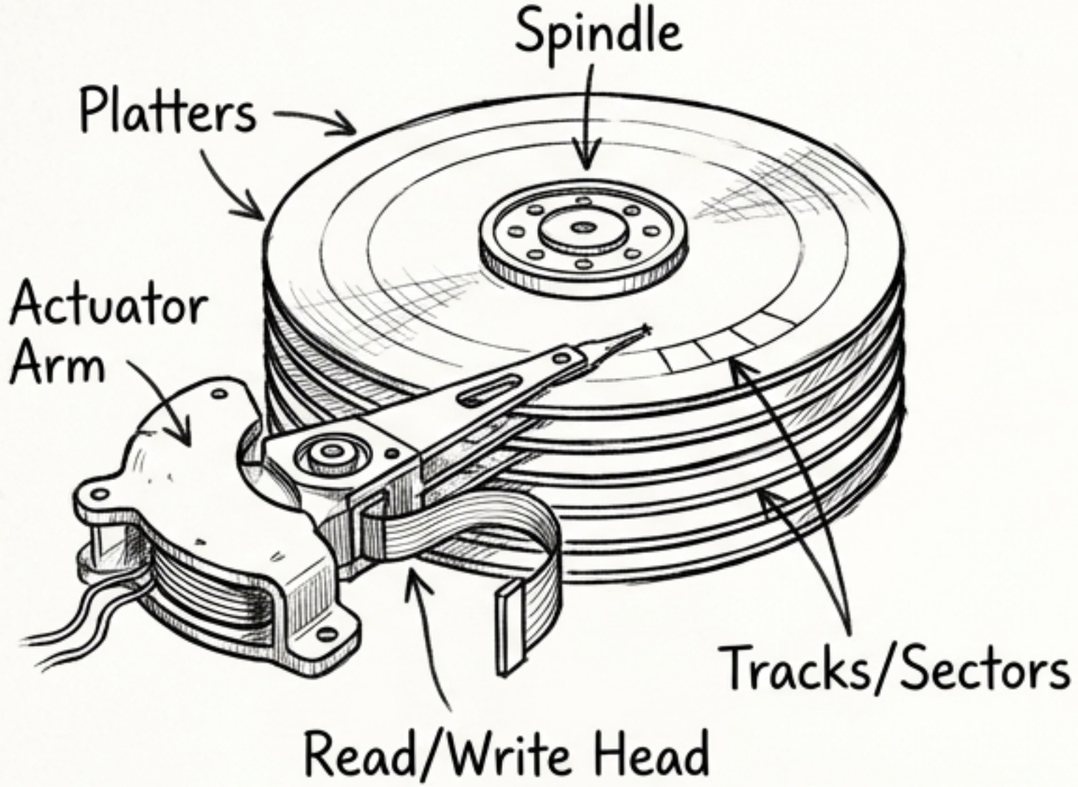
A File System is the bridge between human expectation (named, organized files) and physical reality (a massive, linear array of permanent sectors).

We have to live with exactly what we put on the disk.





The Physical Layer: Magnetic Reality



1 sector = 512B (2^9 Bytes)

1TB Disk = $\frac{10^{12}}{512B} \approx 2^{31}$ sectors

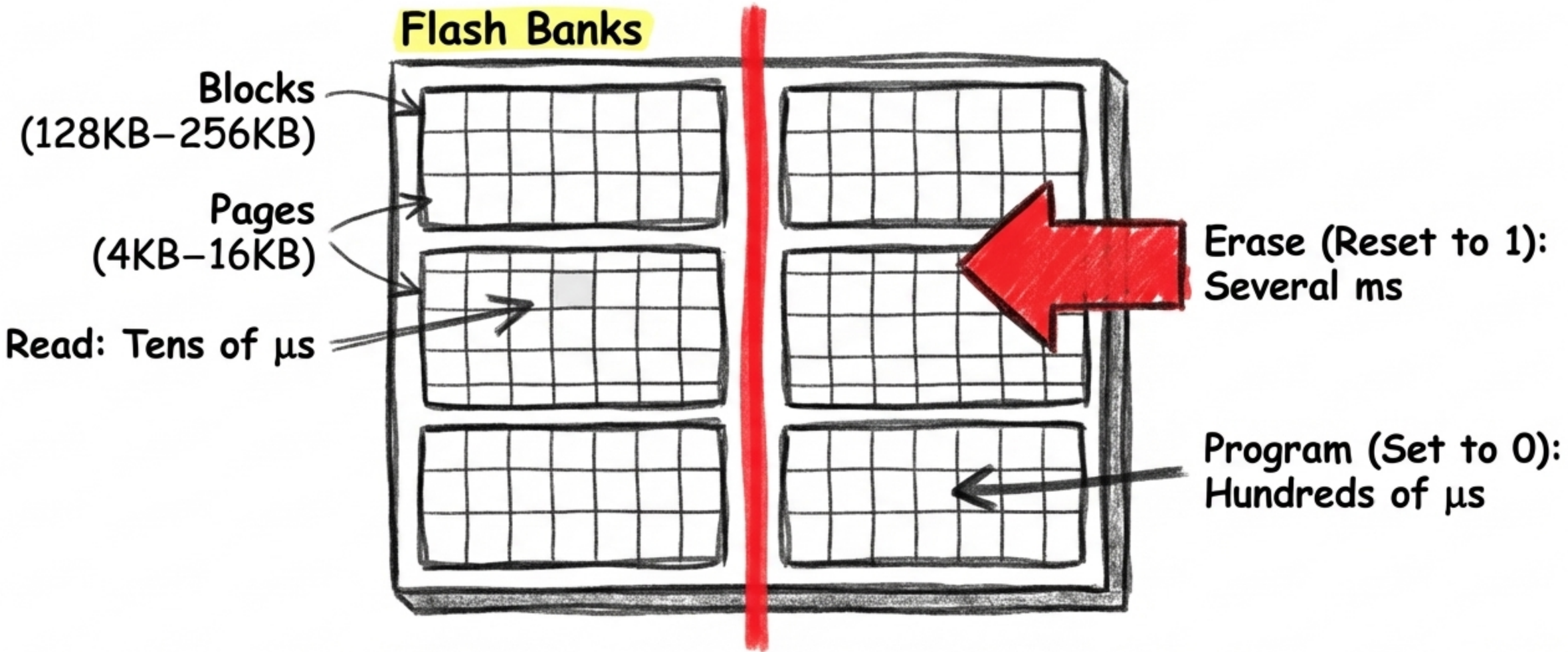
Modern ATA-6 LBA (Logical Block Addressing) uses 48 bits.

$2^{48} \times 512B =$ 144 Petabytes

Key Insight: Hardware failures are guaranteed. Google buys defective, cheap disks intentionally and relies on software fault-tolerance to handle the inevitable crashes. MTBF (Mean Time Between Failures) is just a statistic; software is the safety net.

The Solid State Shift: The Erase-Before-Write Rule

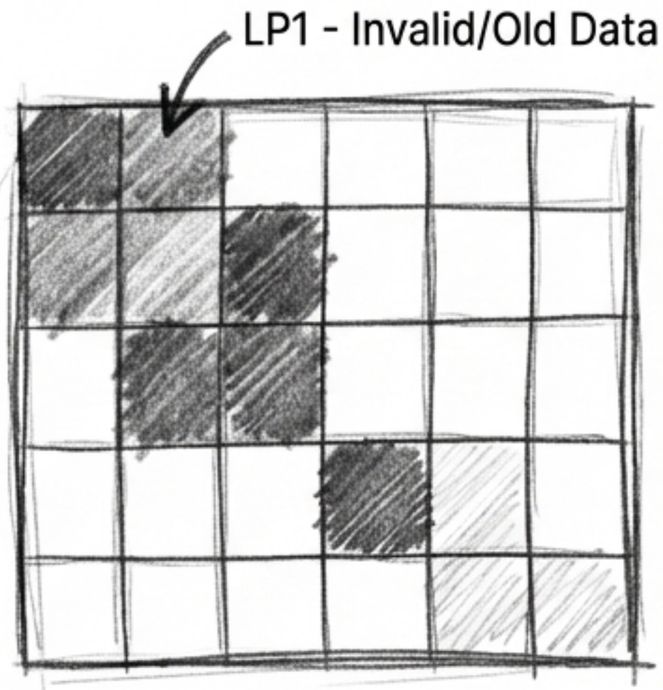
SSDs solve mechanical latency (no seeking), but introduce a new physical quirk. You cannot program a page **twice without** erasing the entire block first. This **physical asymmetry** dictates all modern storage architecture.



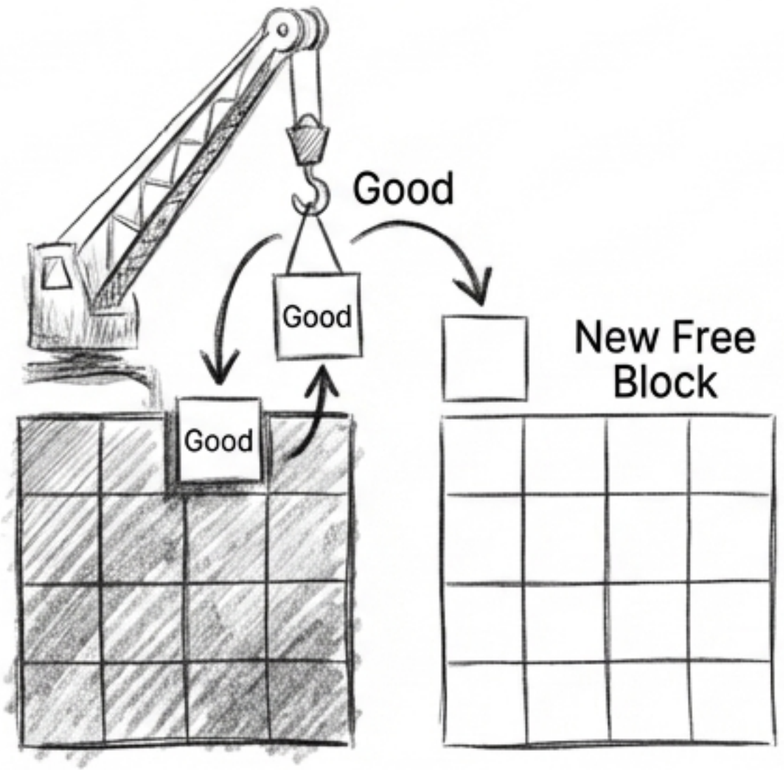
Flash Translation Layer (FTL) & Garbage Collection

To mitigate block wear-out (10k–100k erases), a log-structured FTL intercepts logical writes and appends them to new free pages. It then quietly runs Garbage Collection in the background—moving useful pages and erasing old blocks to free up space.

Step 1: Identify Invalid/Old Data



Step 2: Move Valid Pages



Step 3: Erase Old Block & Update FTL